

Universidad Nacional de Colombia

Sede Bogotá

Curso: Bases de Datos

Proyecto Final — Bases de Datos
Sesión 3: Implementación, Poblamiento y
Consultas
«UniGrades»

Profesor

Cristhian Fernando Lara Espejo

Nombres y códigos de los estudiantes:

Santiago Rafael Puentes Durán: 1032936643

Sergio Gabriel Chaves Mosquera: 1014990992

Cristian Hernández Muñoz: 1141320791

Cesar Samuel Sánchez Romero: 1011100422

Edison Stiven Quintero Motta: 1028561323

Resumen

Este documento presenta la implementación funcional de **UniGrades** sobre el esquema relacional definido en la Sesión 2. Se describe el poblamiento de la base de datos con volumen realista, la implementación de las reglas de negocio como *triggers*, procedimientos almacenados y funciones, y una batería de 15 consultas SQL organizadas por nivel de complejidad creciente que demuestran el valor analítico del modelo. Todos los scripts (01_ddl.sql, 02_dml.sql, 03_plsql.sql y 04_consultas.sql) se ejecutaron de forma secuencial y reproducible sobre una instancia limpia de MySQL 8, sin errores.

Índice

1. Poblamiento de Datos	3
1.1. Volumetría	3
1.2. Estrategia de generación	3
1.3. Casos límite cubiertos	3
2. Reglas de Negocio Implementadas: Triggers	4
2.1. trg_validar_porcentaje_insert / trg_validar_porcentaje_update	4
2.2. trg_actualizar_estado_materia	4
2.3. trg_validar_prerrequisito_diferente / trg_validar_correquisito_diferente . .	4
3. Procedimientos y Funciones	5
3.1. Funciones	5
3.2. Procedimientos	5
4. Batería de Consultas SQL	5
4.1. N1 — Selección y filtrado	6
4.2. N2 — JOINS múltiples	6
4.3. N3 — Agregación	6
4.4. N4 — Subconsultas	6
4.5. N5 — Funciones de ventana	6
4.6. N6 — Vistas	7
5. Pruebas Integrales	7
6. Conclusiones	7

1. Poblamiento de Datos

El script `02_dml.sql` puebla la base de datos inmediatamente después del DDL, sin intervención manual, garantizando la regla de reproducibilidad exigida por el entregable.

1.1. Volumetría

Tabla	Filas	Tipo
universidad	10	Maestra
programa	10	Maestra
tipologia	10	Maestra
materia	390	Maestra
usuario	200	Maestra
semestre	390	Transaccional
materia_usuario	1319	Asociativa (M:N)
componente	3957	Transaccional
nota	4900	Transaccional principal

La tabla transaccional principal (*nota*) supera ampliamente el mínimo de 1 000 filas exigido, y la tabla asociativa (*materia_usuario*) supera el mínimo de 50. Todas las tablas maestras cumplen el mínimo de 10 filas.

1.2. Estrategia de generación

Las tablas maestras (universidades, programas, tipologías y materias) se insertaron con datos reales de instituciones colombianas y códigos de materias existentes. Los usuarios y semestres se generaron con nombres y fechas coherentes, distribuidos entre 2022 y 2026.

Para *materia_usuario*, *componente* y *nota* se implementó un procedimiento temporal (*sp_poblar_transaccional*) que recorre cada semestre registrado, inscribe entre 4 y 6 materias del programa correspondiente, genera 3 componentes por materia (30 %/30 %/40 %) y calcula notas coherentes con un estado objetivo (*aprobada* $\approx$ 82 %, *reprobada* $\approx$ 18 %). El semestre más reciente de cada estudiante se deja en estado *en_curso*, con evaluación parcial, simulando un escenario real de progreso académico.

1.3. Casos límite cubiertos

- Componentes cuya suma de porcentajes llega exactamente a 100.00 (límite superior permitido por el *trigger* de validación).
- Notas en el límite inferior (0.0) y superior (5.0) de la escala.

- Materias en estado *en_curso* con evaluación incompleta (solo un componente calificado).
- Componentes con más de una entrega de nota, demostrando la relación 1:N entre componente y nota.

2. Reglas de Negocio Implementadas: Triggers

Se implementaron cuatro *triggers* en `03_plsql.sql`, todos validando restricciones que no pueden expresarse con CHECK o FOREIGN KEY estándar por depender de múltiples filas o de lógica condicional.

2.1. `trg_validar_porcentaje_insert` / `trg_validar_porcentaje_update`

Regla de negocio: la suma de `componente_porcentaje` de todos los componentes de una misma `materia_usuario` no puede superar 100.

Evento: BEFORE INSERT y BEFORE UPDATE sobre `componente`, FOR EACH ROW.

Caso positivo: insertar un componente cuyo porcentaje, sumado a los existentes, es ≤ 100 — se acepta.

Caso negativo: insertar un cuarto componente que llevaría la suma a 110 — el *trigger* lanza SIGNAL SQLSTATE '45000' y rechaza la operación.

2.2. `trg_actualizar_estado_materia`

Regla de negocio: cuando una materia alcanza el 100 % de su porcentaje evaluado, su estado debe actualizarse automáticamente a *aprobada* o *reprobada* según la nota mínima de la materia.

Evento: AFTER INSERT sobre `nota`, FOR EACH ROW.

Caso positivo: al registrar la última nota que completa el 100 % de evaluación con promedio ponderado $\geq$ nota mínima, el estado cambia a *aprobada* automáticamente.

Caso negativo (control): si la materia ya está *aprobada* o *reprobada*, el *trigger* no vuelve a modificar su estado, respetando la regla de negocio de que un registro cerrado no regresa a *en_curso*.

2.3. `trg_validar_prerrequisito_diferente` / `trg_validar_correquisito_diferente`

Regla de negocio: una materia no puede ser prerrequisito ni correquisito de sí misma.

Evento: BEFORE INSERT sobre materia_prerrequisito y materia_correquisito, FOR EACH ROW.

Caso positivo: insertar (materia_id = 5, prerrequisito_materia_id = 2) — se acepta.

Caso negativo: insertar (materia_id = 5, prerrequisito_materia_id = 5) — se rechaza con SIGNAL SQLSTATE '45000'.

3. Procedimientos y Funciones

3.1. Funciones

fn_promedio_semestre(p_usuario_id, p_semestre_id) — retorna el promedio ponderado por créditos de un estudiante en un semestre específico, excluyendo materias cuya tipología no cuenta para el promedio.

fn_creditos_aprobados_tipologia(p_usuario_id, p_tipologia_id) — retorna el total de créditos aprobados por un estudiante en una tipología determinada, insumo directo para calcular el avance de graduación.

3.2. Procedimientos

sp_resumen_academico_usuario(p_usuario_id) — genera el resumen académico completo de un estudiante: encabezado con promedio global y créditos, y un detalle semestre a semestre obtenido mediante un **cursor explícito** que recorre los semestres del estudiante, acumulando los resultados en una tabla temporal. Incluye manejo de excepciones (CONTINUE HANDLER FOR NOT FOUND para el cursor, EXIT HANDLER FOR SQLEXCEPTION con ROLLBACK y RESIGNAL para errores generales).

sp_inscribir_materia(p_usuario_id, p_materia_id, p_semestre_id, p_mensaje) — encapsula la inscripción de una materia validando, en una transacción, que el semestre pertenezca al usuario, que la materia pertenezca a su programa, que no esté ya aprobada ni ya inscrita en ese semestre, y que todos sus prerrequisitos estén aprobados. Ante cualquier violación hace ROLLBACK y retorna el motivo en el parámetro de salida p_mensaje.

4. Batería de Consultas SQL

Se desarrollaron 15 consultas en 04_consultas.sql, organizadas en los seis niveles exigidos. A continuación se listan las preguntas de negocio que responde cada una.

4.1. N1 — Selección y filtrado

1. ¿Qué materias de 3-4 créditos, sugeridas entre los semestres 2 y 5, contienen “Datos”, “Software” o “Computación” en el nombre?
2. ¿Qué notas aprobatorias fueron registradas entre marzo de 2025 y julio de 2026 en evaluaciones parciales o proyectos?

4.2. N2 — JOINS múltiples

1. ¿Qué materias cursa cada estudiante y a qué programa, universidad y período pertenecen? (INNER JOIN de seis tablas).
2. ¿Qué materias del catálogo nunca han sido cursadas? (LEFT JOIN significativo frente a INNER JOIN).
3. ¿Cuál es el detalle completo de cada calificación — estudiante, materia, componente y período? (INNER JOIN de siete tablas).

4.3. N3 — Agregación

1. ¿Cuál es el promedio, la nota mínima, la máxima y el número de materias evaluadas por programa, año y período?
2. ¿Qué materias presentan al menos cinco intentos y cuál es su tasa de aprobación?
3. ¿Cuántos créditos aprobados acumulan los estudiantes por programa y tipología?

4.4. N4 — Subconsultas

1. ¿Qué estudiantes tienen un promedio global superior al de los estudiantes de su mismo programa? (subconsulta correlacionada).
2. ¿Qué materias no han sido aprobadas por ningún estudiante? (NOT EXISTS).
3. ¿Cuál es el avance total de cada estudiante frente al total oficial de créditos de su programa? (CTE con WITH).

4.5. N5 — Funciones de ventana

1. ¿Cuál es el ranking de estudiantes dentro de cada programa según su promedio global? (DENSE_RANK, ROW_NUMBER).
2. ¿Cómo cambió el promedio de cada estudiante respecto al semestre inmediatamente anterior? (LAG).

4.6. N6 — Vistas

1. ¿Cuántos créditos ha aprobado y cuántos le faltan a cada estudiante por tipología? (reutiliza `v_avance_tipologia`).
2. ¿Cuál es el resumen global y semestral de rendimiento de cada estudiante? (reutiliza `v_promedio_global` y `v_promedio_semestre`).

5. Pruebas Integrales

Se ejecutó la secuencia completa `01_ddl.sql` → `02_dml.sql` → `03_plsql.sql` → `04_consultas.sql` sobre una instancia limpia (`DROP DATABASE` previo), verificando ejecución de extremo a extremo sin errores. Se probaron intencionalmente las siguientes violaciones, todas rechazadas correctamente por los *triggers*:

1. Inserción de un componente que llevaría la suma de porcentajes por encima de 100 — rechazada.
2. Inserción de una materia como prerrequisito de sí misma — rechazada.
3. Inserción de una nota fuera del rango 0.0–5.0 — rechazada por la restricción `CHECK` del DDL.

6. Conclusiones

La implementación física de **UniGrades** confirma que el diseño relacional definido en la Sesión 2 soporta sin fricciones las reglas de negocio más complejas del dominio, en particular la validación de porcentajes acumulados y la actualización automática del estado académico mediante *triggers*. El poblamiento con más de 4 900 registros de `nota` permite que las funciones de ventana y las consultas de agregación produzcan resultados analíticamente significativos, no artificiales.

La batería de 15 consultas demuestra que el modelo responde preguntas reales del dominio académico — ranking de estudiantes, avance por tipología, evolución semestral — validando el valor analítico del esquema de cara a la Sesión 4 (optimización y preparación del *pitch*).